UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

Algoritmos e Estruturas de Dados

A • Exame Época de Recurso • 30 de janeiro de 2023 • 16:00 – 18:00

- Para perguntas com resposta de escolha múltipla, respostas erradas com cotação c e n respostas possíveis descontam $-c / (n - 1)$.
- As respostas às perguntas de desenvolvimento deverão ser as mais detalhadas possíveis, indicando os vários passos intermédios da resolução.
- Na mesa em que está a fazer o exame deve ter apenas lápis/caneta, identificação, este enunciado, e folha de teste.
- Identifique cada folha de teste com o seu nome e número, em letra bem legível.

Grupo I: Perguntas de escolha múltipla

1. (0.5) Considere as seguintes afirmações:

1. O modificador de acesso *private* indica que um método ou atributo é acessível apenas dentro da própria classe.
2. Quando não se especifica um modificador de acesso, o Java considera por omissão o modificador de acesso *default*.
3. O modificador de acesso *default* indica que um método ou atributo é acessível dentro da própria classe e por outras classes que herdem desta classe.
4. O modificador de acesso *public* indica que um método ou atributo é acessível dentro de todas as classes que pertençam à mesma package (onde o método/atributo foi definido).

Quais das afirmações acima são verdadeiras?:

a) 1 e 2
b) 1 e 3
c) 1 e 4
d) 1, 2 e 3
e) todas
f) nenhuma

2. (0.5) Considere o algoritmo de ordenação Shellsort usando a sequência de Knuth aplicado ao seguinte array [8, 1, 0, 3, 7, 6, 4, 9, 2, 5]. Indique qual das seguintes sequências corresponde a uma sequência válida e pela ordem correta do estado do array após operações de h-sorting. Assuma que h inicial = 4.

a)
[2, 1, 0, 3, 7, 5, 4, 9, 8, 6]
[0, 1, 2, 3, 4, 5, 7, 6, 8, 9]

b)
[2, 1, 4, 3, 7, 6, 0, 5, 8, 9]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

c)
[2, 1, 4, 3, 7, 6, 0, 5, 8, 9]
[0, 1, 2, 3, 7, 6, 4, 5, 8, 9]

d)
[2, 1, 0, 3, 7, 5, 4, 9, 8, 6]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

3. (0.5) Considere o seguinte código:

```
int sum = 0;
for(int i = n; i >0 n; i/=2)
{
    for(int j = 0; j < n-1 ; j++)
    {
        sum++;
    }
}
```

Indique a complexidade temporal assimptótica do código apresentado.

a) $O(n^3)$
b) $O(n \log n)$
c) $O(\log n)$
d) $O(n^2)$
f) $O(n)$

4. (0.5) Depois de ter implementado um determinado algoritmo, decidiu executar ensaios de razão dobrada para tentar estimar de forma empírica a complexidade temporal assimptótica do algoritmo. Os resultados obtidos foram os seguintes:

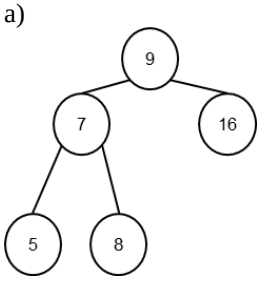
i	n	time(ms)	estimated r
…			
4	2000	0,6	3,6
5	4000	2,3	3,833
6	8000	10,16666	4,420
7	16000	38,76666	3,813
12	32000	130,6333	3,369
13	64000	501,3666	3,837

Indique a complexidade temporal assimptótica estimada, tendo em conta os resultados obtidos:

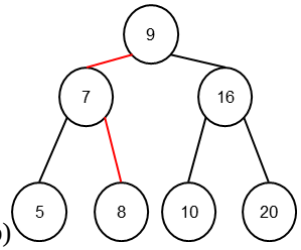
a) $O(n^3)$
b) $O(\log n)$
c) $O(1)$
d) $O(n)$
e) $O(n^2)$

5. (0.5) Qual das seguintes árvores é uma árvore red-black válida?

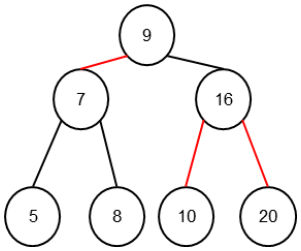
a)



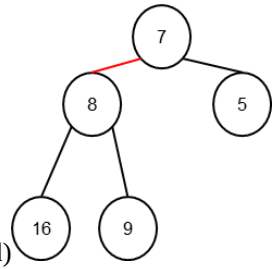
b)



c)



d)



e) Nenhuma das árvores apresentadas é uma árvore red-black válida

6. (0.5) Considere uma árvore de pesquisa binária que se encontra inicialmente vazia. Seguidamente são inseridos nesta árvore, sequencialmente, e por ordem crescente, todos os números inteiros entre 1 e *n*. Após esta inserção, indique qual a aproximação tilde mais correta para o tempo de pesquisa de um nó *k*, que **não faz parte da árvore**.

a) $\sim n$
b) $\sim\sim \frac{n}{2}$
c) $\sim \log n$
d) $\sim 2 \log n$

Grupo II: Perguntas de desenvolvimento

7. (1.5) Escreva um método estático chamado multiplicarSimNao na linguagem Java, que dado um array de inteiros a, retorna um inteiro que corresponde à multiplicação de todos os índices pares do array. Se o array recebido estiver vazio, deve ser retornado 1.

Por exemplo, se a=[1,2,3,4,5], então a função devolve 1*3*5 = 15. Se a=[4,7,4,2,3,1] a função devolve 4*4*3=48.

8. (1.5). Uma vantagem de uma lista duplamente ligada é que permite encontrar um item da lista através de um índice em metade do tempo de uma lista simplesmente ligada (~n/2). Por exemplo, se o índice se encontrar mais perto do fim da lista, começamos pelo último nó, e vamos retrocedendo até encontrar o índice desejado. Outra vantagem é que usando um pequeno truque – um *array* de 2 ponteiros em que um dos ponteiros aponta para um lado, e o outro aponta para a direção contrária - podemos implementar uma operação de inversão de uma lista em tempo constante. Considerando a seguinte implementação parcial de uma lista duplamente ligada:

```
public class DoubleLinkedList<Item> {
    private Node<Item> first;
    private Node<Item> last;
    private int forwardDirection;
    private int backwardDirection;
    private int size;

    public DoubleLinkedList()
    {
        this.first = null;
        this.last = null;
        this.size = 0;
        this.forwardDirection = 1;
        this.backwardDirection = 0;
    }
```

```
class Node<Item>
{
    Item item;
    Node<Item>[] nodePointers;

    Node(Item item)
    {
        this.item = item;
        this.nodePointers =
                (Node<Item>[]) new Object[2];
    }
```

```
    public Item get(int index)
    {
        if(index >= this.size) return null;
        Node<Item> current;
        if(index>this.size/2)
        {
            //when the index is closer to the end, start from the last node
            //and go back until you find the right node
            current = this.last;
            for(int i = size-1; i>index; i--)
            {
                current = current.nodePointers[backwardDirection];
            }
        }
        else
        {
            //when the index is closer to the start, start from the first node
            //and move forward until you find the right node
            current = this.first;
            for(int i = 0; i < index; i++)
            {
                current = current.nodePointers[forwardDirection];
            }
        }

        return current.item;
    }

    public void reverse()
    {
       //TODO: Implement
    }
```

Implemente o método reverse() de forma a que a inversão completa da lista tenha uma complexidade temporal de O(1).

9. (2.0). Considere a classe *SmartList* implementada no Projeto P2, em que sempre que é feita uma pesquisa sobre um elemento na lista, e esse elemento é encontrado, é aplicada a técnica de *MoveToFront* ou a técnica de *Transpose*.

a) (1.0) Indique no que consiste a técnica *MoveToFront*, e a técnica *Transpose*.

b) (1.0) Qual a vantagem de uma *SmartList* face a uma lista ligada tradicional?

10. (1.5) Considere o que acontece à fila q à medida que o seguinte código é executado:

```
QueueList<String> q = new QueueList<String>();
StackList<String> stack = new StackList<String>();
q.enqueue("AED");
q.enqueue("a");
q.enqueue("nota");
q.enqueue("boa");
q.enqueue("ter");
q.enqueue("Vou");
while(!q.isEmpty())
{
    stack.push(q.dequeue());
}
while(!stack.isEmpty())
{
    q.enqueue(stack.pop());
}
while(!q.isEmpty())
{
    System.out.print(q.dequeue() + " ");
}
```

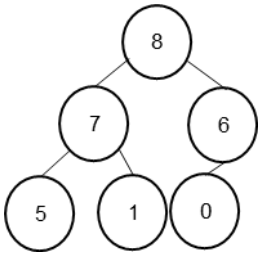
Desenhe o estado da fila imediatamente antes do início de cada ciclo *while*, e indique a sequência impressa no ecrã.

11. (2.0) Considere o algoritmo de ordenação *MergeSortBottomUp* aplicado ao seguinte array:

a=[5, 7, 3, 8, 1, 6, 4, 9, 2,10,0]

Execute o algoritmo até que o *array* esteja completamente ordenado, indicando os subconjuntos e as operações de *merge* em cada iteração do ciclo exterior do algoritmo.

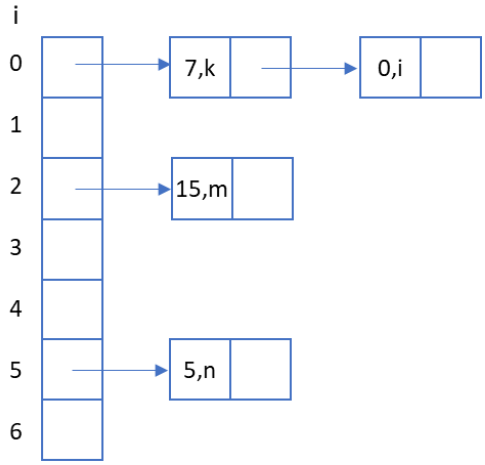
12. (2.5) Considere o seguinte max-heap:



a) (1.5) Desenhe a representação em árvore do *max-heap* obtido quando se insere os elementos 11 e 9 no heap. Desenhe os principais passos intermédios.

b) (1.0) Desenhe a representação em árvore do *max-heap* obtido quando se remove o maior elemento do ***heap* original**. Desenhe os principais passos intermédios.

13. (3.0) Considere a seguinte tabela de dispersão com resolução de colisões por encadeamento separado (separate chainning) de dimensão m = 7, e com função de dispersão h1(k) = k % m.



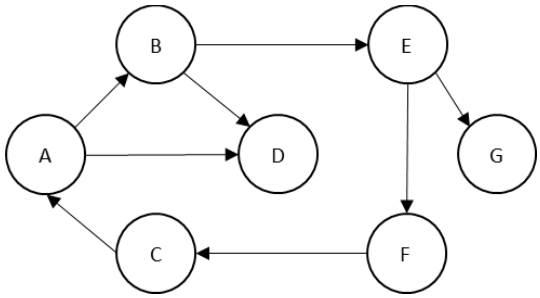
A tabela encontra-se inicialmente preenchida com os pares chave,valor: <7,k>,<15,m>,<5,n> e <0,i>. Uma célula vazia representa uma lista vazia ou um ponteiro *null*.

a) (1.0) Desenhe a tabela que se obtém apagando as chaves 7 e 5.

b) (2.0) Desenhe a tabela que se obtém, após inserir os seguintes pares <chave,valor> na **tabela original**. Indique o valor de h1 calculado para cada chave.
<4,a>,<7,b>,<2,c>,<16,d>,<13,e>

14. (1.0) Qual é a diferença entre um grafo dirigido e um grafo não dirigido?

15. (2.0) Considere o seguinte grafo.



a) Faça uma procura em profundidade primeiro sobre o grafo, considerando que a procura se inicia a partir do vértice A. Considere que os vértices adjacentes de um vértice são visitados (ou testados) por ordem alfabética. Indique a pré-ordem (ordem em que os vertices são visitados), e a pós-ordem (ordem pela qual é feito o retrocesso de cada vértice).